

Code Explanation

Code Explanation: AWS Glue PySpark ETL Job Test Suite
This is a comprehensive test suite for an AWS Glue PySpark ETL job that processes customer and order data with SCD Type 2 (Slowly Changing Dimension) logic.---

****Test Structure Overview****

****1. Test Fixtures (Setup Components)****

****Spark Session Fixture****

```
@pytest.fixture(scope="session")def spark():
```

- Creates a local Spark session for testing- Uses `local[2]` mode (2 threads) for parallel processing- Makes Spark available globally via `\_\_builtins\_\_`- Cleans up after all tests complete##### ****Sample Data Fixtures****
The suite creates several test datasets:
- **sample_customer_data**: Clean customer records- 4 customers with ID, name, email, and region- No nulls or duplicates
- **sample_customer_data_with_nulls**: Dirty customer data- Contains "Null" string values- Contains actual NULL values- Contains duplicate records- Used to test data cleaning logic
- **sample_order_data**: Order transactions- 4 orders linked to customers via CustId- Contains order dates and amounts as strings
- **sample_previous_customer_state**: Historical SCD Type 2 data- Contains existing customer records with SCD columns (IsActive, StartDate, EndDate, OpTs)- Used to test change detection and versioning logic---

****2. Configuration Tests****

****test_get_job_parameters****

****Purpose****: Validates job configuration loading
****Verifies****:
- Configuration structure contains required sections (inputs, outputs, catalog, hudi, processing, scd_type2)- Customer input path: `s3://adif-sdlc/sdlc\_wizard/customerdata`- Order input path: `s3://adif-sdlc/sdlc\_wizard/orderdata`- Catalog database: `sdlc\_wizard\_db`- Table definitions exist for customer, order, and ordersummary---

****3. Initialization Tests****

****test_initialize_spark_contexts****

****Purpose****: Tests Spark and Glue context initialization
****Validates****:
- SparkSession is created- GlueContext is initialized- Job object is created- Job name is set to "test_job"---

****4. Schema Definition Tests****

****test_get_customer_schema****

****Purpose****: Validates customer schema structure
****Checks****:
- Schema has 4 fields: CustId, Name, EmailId, Region- CustId is non-nullable (primary key)- All fields are StringType- Field order is correct

****`test\_get\_order\_schema`****

****Purpose****: Validates order schema structure****Checks****:- Schema has 4 fields: OrderId, CustId, OrderDate, Amount- Field types and nullability are correct---

****5. Data Transformation Tests****

****`test\_normalize\_columns`****

****Purpose****: Tests column name standardization****Process****:- Converts column names to lowercase- "CustId" → "custid"- "EmailId" → "emailid"****Validation****:- All lowercase column names exist- Original mixed-case names are removed

****`test\_remove\_nulls\_and\_duplicates`****

****Purpose****: Tests data quality cleaning****Input****: 5 rows with issues- 1 row with "Null" string- 1 row with actual NULL value- 1 duplicate row****Expected Output****: 2 clean rows****Validation****:- No NULL values remain- No "Null" string values remain- Duplicates are removed---

****6. Data Reader Tests****

****`test\_data\_reader\_csv`****

****Purpose****: Tests CSV file reading with mocking****Process****:- Mocks the internal `\_read\_csv` method- Calls `read\_data\_safe` with CSV format- Returns sample customer data****Validation****:- DataFrame is returned- Row count is correct (4 rows)- Mock method was called once

****`test\_data\_reader\_parquet`****

****Purpose****: Tests Parquet file reading****Similar to CSV test but for Parquet format****

****`test\_data\_reader\_error\_handling`****

****Purpose****: Tests graceful error handling****Process****:- Mocks `\_read\_csv` to raise an exception- Calls `read\_data\_safe`****Expected****: Returns `None` instead of crashing---

****7. Data Writer Tests****

****`test\_data\_writer\_parquet` / `test\_data\_writer\_csv`****

****Purpose****: Tests data writing capabilities****Process****:- Mocks internal write methods- Calls `write\_data\_safe`****Validation****:- Returns `True` on success- Mock method called once

****`test\_data\_writer\_error\_handling`****

****Purpose****: Tests write error handling****Expected****: Returns `False` on failure (doesn't crash)---

****8. Catalog Integration Tests****

****`test\_write\_to\_catalog`****

****Purpose****: Tests Glue Data Catalog registration****Process****:- Mocks the DataWriter- Calls `write\_to\_catalog` with database, table, and path parameters****Validation****:- Returns `True` on success- Data is written to specified S3 path---

****9. SCD Type 2 Change Detection Tests****

****`test\_detect\_customer\_changes\_initial\_load`****

****Purpose****: Tests change detection with no historical data****Scenario****: First-time data load****Expected****: All records are considered "changed" (new)

****`test\_detect\_customer\_changes\_with\_changes`****

****Purpose****: Tests change detection with existing data****Scenario****: - C001's email changed from "john.old@example.com" to "john@example.com"- C003 and C004 are new customers****Expected****: At least 1 changed record detected---

****10. SCD Type 2 Application Tests****

****`test\_apply\_scd\_type2\_initial\_load`****

****Purpose****: Tests SCD Type 2 for initial data load****Process****:- Applies SCD Type 2 logic with no previous state- Adds SCD columns: IsActive, StartDate, EndDate, OpTs****Validation****:- All records have SCD columns- All records are marked as active (IsActive = True)- EndDate is NULL for active records

****`test\_apply\_scd\_type2\_with\_changes`****

****Purpose****: Tests SCD Type 2 with historical data****Process****:- Compares current data with previous state- Creates new versions for changed records- Expires old versions (sets EndDate, IsActive = False)****Expected****:- More records than input (due to versioning)- Old versions are expired- New versions are active---

****11. Business Logic Tests****

****`test\_calculate\_customer\_aggregate\_spend`****

****Purpose****: Tests customer spending aggregation****Business Logic****:- Groups orders by customer and date- Sums order amounts per customer per day****Example Calculation****:- Customer C001 on 2024-01-01: - Order O001: \$100.50 - Order O002: \$200.75 - ****Total: \$301.25****Validation****:- Aggregated DataFrame has correct columns (custid, orderdate, totalspend)- Calculations are accurate (within 0.01 tolerance)---

****12. Hudi Integration Tests****

`**test_write_hudi_table**`

Purpose: Tests Apache Hudi table writing with fallback
Process: - Adds SCD columns to sample data- Attempts to write as Hudi table- Falls back to Parquet if Hudi unavailable
Validation: - Returns `True` on success (either format)- Handles Hudi unavailability gracefully---

13. Integration Tests

`**test_main_integration**`

Purpose: Tests the main ETL pipeline with mocked I/O
Mocked Operations: - CSV reads (customer and order data)- Parquet reads (previous state)- Parquet writes (output)- Hudi writes
Validation: - Main function executes without exceptions- All components integrate correctly

`**test_end_to_end_pipeline**`

Purpose: Tests complete data flow through all transformations
Pipeline Steps: 1. **Normalize**: Convert column names to lowercase 2. **Clean**: Remove nulls and duplicates 3. **Apply SCD Type 2**: Add versioning columns 4. **Aggregate**: Calculate customer spending
Validation: - Each step produces valid output- Data flows correctly between steps- Final output has expected structure and content---

Key Testing Patterns Used

1. **Mocking**: Prevents actual S3 reads/writes during testing 2. **Fixtures**: Reusable test data across multiple tests 3. **Isolation**: Each test is independent 4. **Validation Layers**: Tests configuration, transformations, and integration separately 5. **Error Handling**: Tests both success and failure scenarios---

Test Execution

Run with:

```
pytest test_job.py -v
```

The `-v` flag provides verbose output showing each test result.